

Web & Mobile Developer

Ju-cheol Park

Jukrap

I'm Ju-cheol Park, a web and mobile developer. I build interfaces with React and TypeScript and have also worked on APIs and mobile integrations.

Role

Web & Mobile Developer

Experience

Web UI development, API development, Android integration

Selected interfaces, implementation decisions, and outcomes from my work.

Email

jukrap628@gmail.com

Alternate email

valurauta628@gmail.com

GitHub

github.com/jukrap

Blog

valur.tistory.com/

LinkedIn

www.linkedin.com/in/jukrap/

Website

jukrap.vercel.app/en

Experience

Work at Triphos

Built the settlement ERP across React interfaces, Spring Boot APIs, and database processing. Developed logistics screens and the complete React frontend for stock administration using MSW mock APIs. I also implemented persistence and access checks, and connected file attachments and label printing to Android apps.

Company / period

Role

Triphos Co., Ltd. 2026.02 ~ Present

Web & Mobile Developer

Selected work

Settlement ERP Platform

Built across the settlement ERP, from React interfaces to Spring Boot APIs and database processing. Implemented shared editing tables that retain input after failed saves and Excel imports that prevent duplicate records.

Logistics Operations Web and Printing App

Built React and TypeScript interfaces spanning search, reservations, Excel imports, and label-print requests. Loaded pages and Excel code on demand, reducing the initial JavaScript entry from 2,405.50 kB to 616.59 kB, about 74%.

Stock Operations Admin Web

Built the complete React frontend for a stock administration web app using MSW mock APIs. Created shared tables, filters and modals, with column pinning, resizing and reordering, and state handling for data queries and editing.

Further work appears on page 7 and the Work page.

Selected work 01

Settlement ERP Platform

Built across the settlement ERP, from React interfaces to Spring Boot APIs and database processing.

Period

2026.07 ~ 2026.09

Scope

Editable tables and Excel imports /
Billing and settlement history

Technologies

React, TypeScript, Tailwind CSS, Vite, React Router, Zustand, Spring Boot, MyBatis, MariaDB, Vitest

Context

Multi-row edits and Excel imports needed to avoid lost input and duplicate writes. Changes to assignments also had to preserve finalized settlement amounts.

Implementation

Editing tables that retain input

Built shared editing tables with keyboard navigation and selection dialogs, retaining drafts after failed saves.

From Excel preview to persistence

Revalidated previewed data before saving and handled rollback and retries to prevent partial or duplicate finalization.

History and audit records

Separated current assignments from historical transactions. Implemented recording of business changes and login results with an administrator query view.

Outcome

Connected table editing and bulk imports to a common save flow, preserved existing amounts after assignment changes, and delivered the implementation to the development environment.

Shared editable tables

Draft preservation

Inline editing, selection dialogs, keyboard navigation and failed-save handling

Excel batch import

No duplicate or partial imports

Roll back failed batches and return the original result on retry

Assignment changes

Settlement history retained

Separate current assignments from original recipients and amounts

Selected work 02

Logistics Operations Web

Built React and TypeScript interfaces spanning search, reservations, Excel imports, and label-print requests.

Period

2026.04 ~ 2026.06

Scope

Operations / Performance

Technologies React, TypeScript, Vite, React Router, TanStack Query, Zustand, XLSX, Vitest

Context

The initial bundle included multiple pages and heavy Excel code that users did not need on their first screen.

Implementation

Shared interfaces and API integration

Built shared tables, forms, and modals, connecting server APIs to workflows from search and reservations through Excel previews and print requests.

Code splitting informed by bundle analysis

Identified code that was unnecessary on entry and loaded pages and Excel libraries only when needed.

Trade-off

Tried splitting shared authentication and API initialization but did not adopt it because it added loading on the first interaction. Kept on-demand loading for pages and Excel code.

Outcome

Reduced the initial JavaScript entry from 2,405.50 kB to 616.59 kB, about 74%, comparing output sizes on the same build basis.

Initial JavaScript entry

2,405.50 → 616.59 kB

About 74% smaller on the same build basis

Gzip-compressed size

815.10 → 204.38 kB

About 75% smaller for the same initial entry

Selected work 03

Stock Operations Admin Web

Built the complete React frontend for a stock administration web app using MSW mock APIs.

Period

2026.02 ~ 2026.03

Scope

Stock operations / admin screens

Technologies React, TypeScript, Vite, React Router, TanStack Query, Zustand, MSW

Context

List, search, detail, and editing screens needed consistent components and clearer ownership of state.

Implementation

Reusable tables, filters, and modals

Built list, search, detail, and create/edit screens with shared components, including pinned columns and column resizing.

Separate query data and UI state

Queried MSW mock responses with TanStack Query while managing filters, selection, and modal state separately.

Outcome

Built query and edit screens against MSW mock APIs, reusing shared components and separating query data from interaction state.

Pinning, resizing and reordering

Shared table controls

Consistent column controls and state display across screens

Frontend development environment

MSW mock responses

Developed data display and UI state without a real backend connection

Related work / Android

Mobile Output Bridge App

Connected web label-print requests to an Android app and Bluetooth printers.

Period

2026.04 ~ 2026.06

Scope

WebView / Android native module

Technologies Expo, React Native, Expo Router, WebView, Android native module, TypeScript, Bluetooth

Context

Web print data needed to reach a native device SDK while the interface handled permissions, connection, and cancellation.

Implementation

From web requests to hardware

Passed print data through a native bridge and implemented device selection, connection, and print calls as a single flow.

Permissions and cancellation

Traced SDK discovery calls on Android 16/API 36 and corrected cancellation and BLUETOOTH_SCAN, BLUETOOTH_CONNECT permission handling.

Outcome

Verified Bluetooth connection and physical label output on an Android device.

output boundary

WebView → native

web payload separated from device output commands

device output

real device check

verified against Android device behavior, not only emulator flow

Experience

Additional work

Further feature development and maintenance across web and mobile.

01	Member and Inquiry Management Web and App	Period 2026.09	Platform Web / Android
Implementation			
Extended an existing solution with member search, editing, merging, inquiries, and attachments. Integrated file selection and image previews in the Android app.			
Technologies Spring MVC, Java, MyBatis, Vue 2, MariaDB, Android, Kotlin			
02	Parcel Delivery App	Period 2026.08	Platform Web / Android
Implementation			
Maintained maps and offline save/retry flows, restoring queued requests after app restarts.			
Technologies React, TypeScript, React Native, Expo, Spring Boot, SQLite			
03	Legacy Logistics Customer Support Web	Period 2026.08	Platform Web
Implementation			
Implemented distinct messages for parsing, input, and network errors in Excel imports and checked them locally.			
Technologies Java, Spring MVC, JSP, MyBatis, jQuery, Apache POI			
04	Daily Information App	Period 2026.06 ~ 2026.07	Platform Hybrid
Implementation			
Extended weather, air quality and alert features with public-data APIs. Deployed core-first loading, server caching and shared results for concurrent requests to reduce repeated API calls.			
Technologies Spring MVC, JSP, jQuery, Java, Android			
05	Chart Preview Page	Period 2026.03 ~ 2026.04	Platform Web
Implementation			
Built chart settings panels, data connections, and previews as part of a website that was under development.			
Technologies React, TypeScript, Vite, Chart.js, WebGL/GLSL (OpenGL-style), OGL (WebGL library), MSW, Vitest			

Project 01

C. Donghae

Built a map-based web service for Donghae Line passengers to find live transport and nearby information.

Period

2024.10.04 ~ 2024.10.06

Role

Frontend developer / team of three

Technologies TypeScript, React, Next.js, Tailwind CSS, Google Maps, Storybook, AWS, GitHub Actions, Docker



Context

The team had 72 hours to combine train, weather, and local-place data in one map with mobile interactions.

Implementation

Maps and API integration

Worked with two backend developers using Swagger documentation to connect APIs and build the main Google Maps screens.

Mobile map interaction

Built a bottom sheet over the map that expands or collapses according to drag distance and velocity.

Outcome

Won the Busan Technopark President Award at DIVE 2024, placing third in the challenge track.

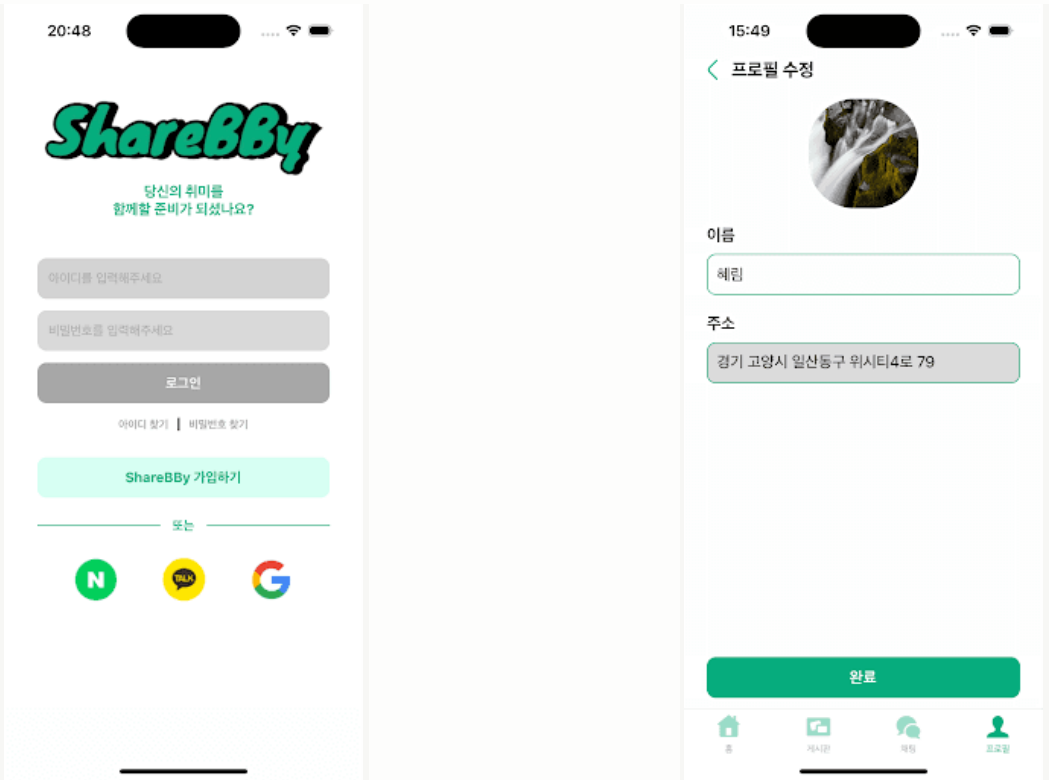
Project 02

ShareBBy

Developed Android support and community features for an app that helps people share hobbies and find companions.

Period	Role
2024.04 ~ 2024.06	React Native and Firebase development

Technologies JavaScript, React Native, Firebase, Faster Image



Context

Screens initially built for iOS needed Android support, and stale image caches prevented updated post images from appearing.

Implementation

Community features

Built post and comment editing, location filters, sorting, pull-to-refresh, and infinite scrolling.

Data and image handling

Documented Firebase relationships in an ERD, reproduced the caching problem, and adopted a maintained replacement library.

Outcome

Delivered Android screens and community features. The team released the app on the App Store in 2024.

GitHub

github.com/jukrap/archived-rn-ShareBBy

Project 03

AI Agent Playbook

A personal developer tool combining a CLI, skills, templates, and checks to carry project rules and history across agent sessions.

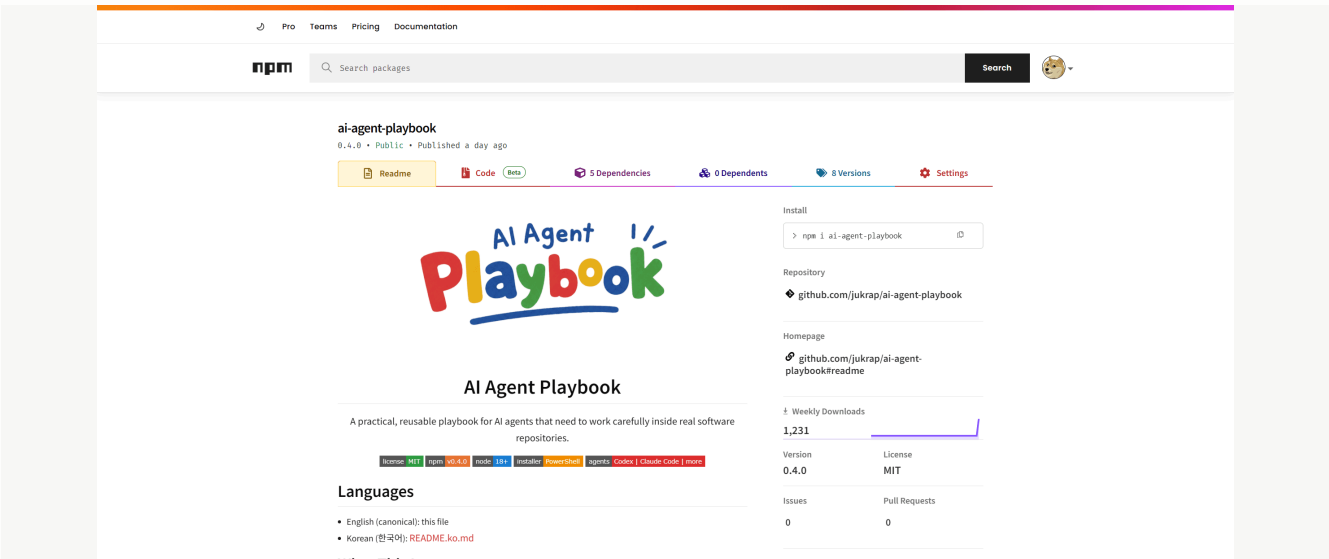
Period

2026.06 ~ Present

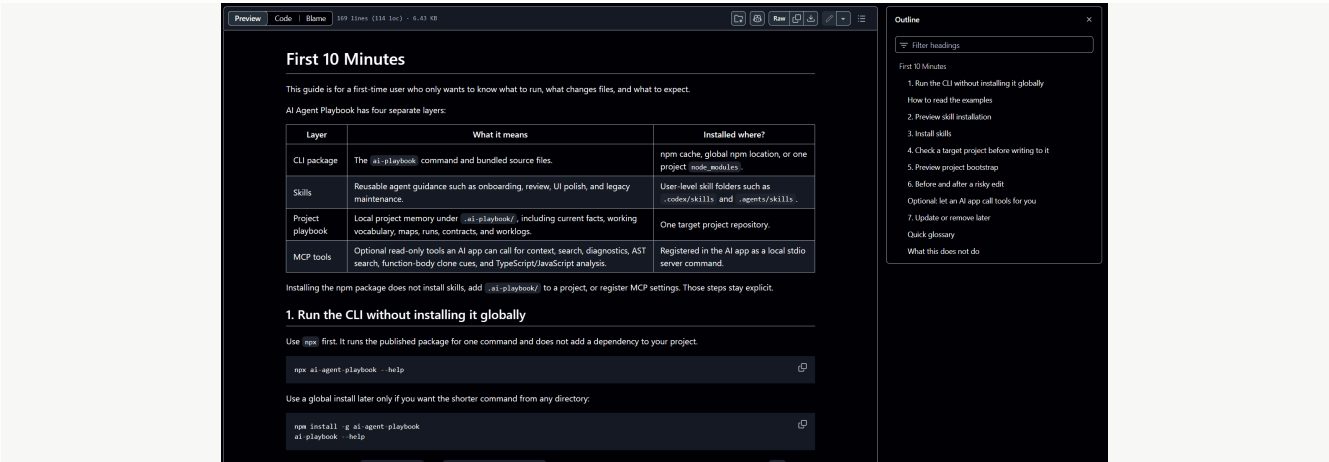
Role

Solo developer

Technologies JavaScript, Node.js, GitHub Actions



npm package



Installation and setup

Context

Scattered rules and records led agents to rediscover the same material, while file-changing commands were difficult to assess before execution.

Implementation

Reusable rules and project records

Separated reusable skills and templates from repository-specific project memory.

Previewing changes

Built a CLI and read-only MCP tools, with dry-run previews for commands that modify files.

Outcome

Published the tool as an npm package and GitHub repository.

GitHub

github.com/jukrap/ai-agent-playbook

Website

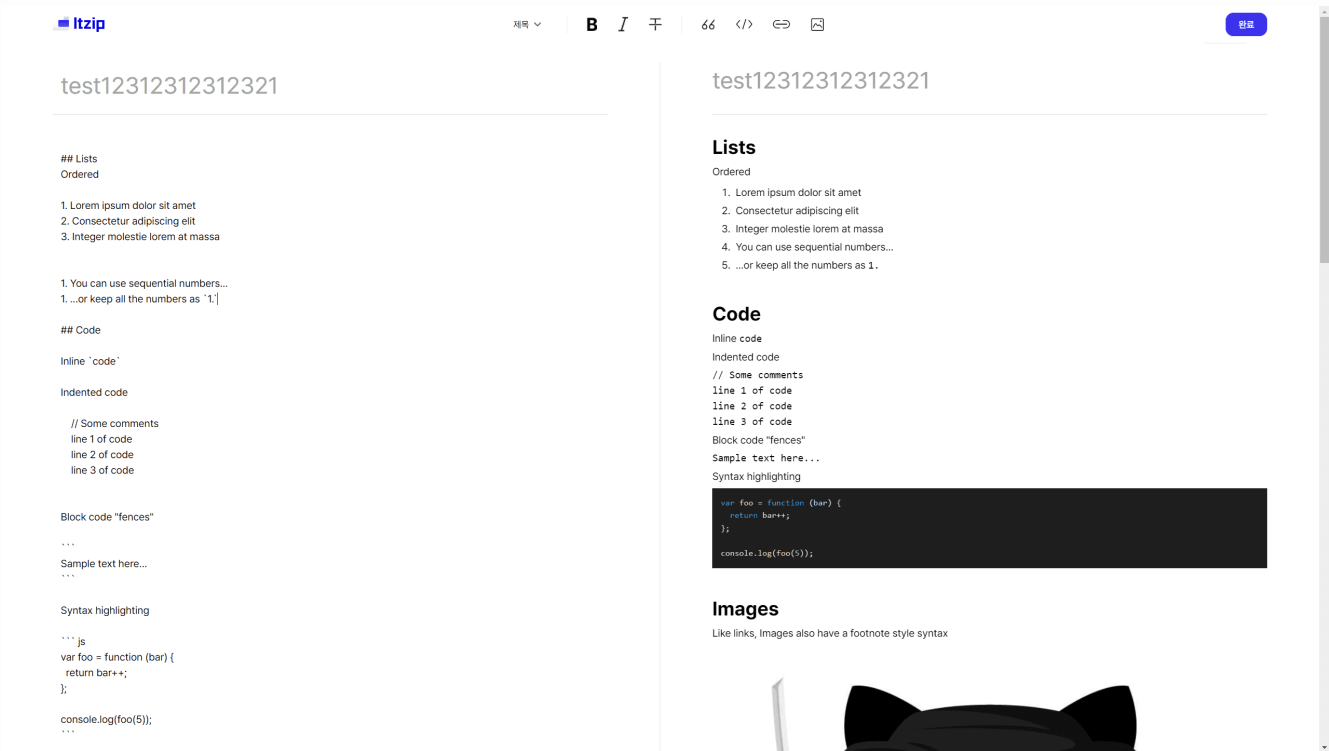
www.npmjs.com/package/ai-agent-playbook

Project 04

Itzip

Built blog and Markdown editing screens in a 15-person team, adding tests and error monitoring to the development workflow.

Period	Role
2024.07 ~ 2025.06	Frontend team lead / DevOps
Technologies TypeScript, React, Next.js, Tailwind CSS, React Spring, Jotai, Jest, Prisma, Storybook, Postman, Sentry, AWS, Jenkins, Docker	



Markdown editing and preview

Context

The team needed a shared way to inspect component behavior and errors after deployment while building the editor.

Implementation

Markdown editing and previews

Implemented live previews and project-specific Markdown syntax, splitting heavy components with dynamic imports.

Tests and error monitoring

Tested key behavior with Jest, documented components in Storybook, and configured Sentry for post-deployment errors.

Outcome

Delivered blog and editor screens with unit tests, component documentation, and error monitoring.

GitHub

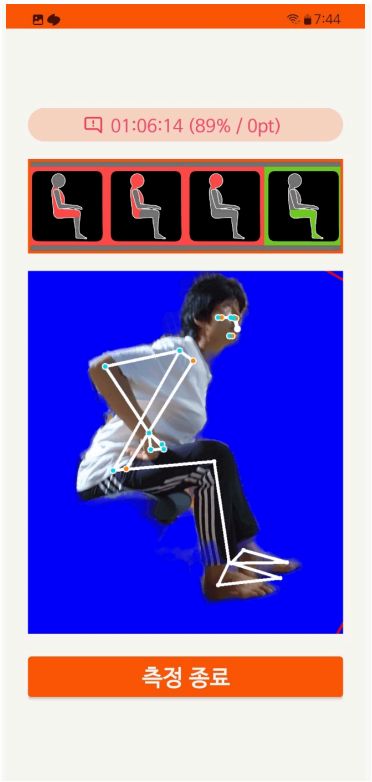
github.com/ITZipProject/itzip_frontend

Project 05

Posture Teacher

An Android app that classifies posture from body landmarks in camera frames and records exercise time.

Period	Role
2022.07 ~ 2023.05	Android developer / team of two
Technologies Java, Android, Jetpack, MediaPipe, SQLite	



Posture analysis and timing records

Context

Continuous frame analysis needed to remain separate from screen input, and MediaPipe had to be built for the Android project.

Implementation

MediaPipe integration and posture detection

Built a MediaPipe AAR on Ubuntu and used landmark angles and lengths to classify posture and measure hold times.

Separate frame analysis

Moved analysis to a separate thread so camera processing and screen input did not block each other.

Outcome

During the project, frame-processing FPS was 5 – 10 times that of the OpenCV implementation, measured as frame-processing throughput.

GitHub
github.com/jukrap/archived-Posture-Teacher

Profile

Skills, education, and awards

Skills

Languages

TypeScript, JavaScript, Java

Mobile / hybrid

React Native, Expo, Android, Gradle

Tools & Data

GitHub Actions, Jenkins, Docker, MariaDB, SQLite

Web

React, Next.js, Vite, TanStack Query, Zustand

Testing / monitoring

Vitest, MSW, Storybook, Sentry

Education

Gyeongsang National University / Computer Science

Graduated 2023.02

GPA 3.64 / 4.5

Programmers Devcourse

2023.12 ~ 2024.05

Cloud Application Engineering, React and React Native

More

Work

jukrap.vercel.app/en/work

Projects

jukrap.vercel.app/en/projects

Awards

DIVE 2024 Global Data Hackathon

2024.10

Busan Technopark President Award / third in the challenge track

Gyeongnam Software Competition

2021.10

ESD HotDeal / Top Excellence Award